

Soccer TIPS Engineering Department

Document: Layer 2_v0.1 General Developer Handoff Notes

General Developer

Layer 2 v0.1 Feature Engineering Foundation Implementation Handoff Notes

Created after Lead Developer acceptance of the Layer 2 v0.1 Feature Engineering Foundation checkpoint.

1. Current Layer 2 v0.1 Status

- Layer 2 v0.1 is accepted as a successful feature-engineering foundation checkpoint.
- The active provider for Layer 2 v0.1 is SkillCorner because it has validated phases_of_play data with in-possession and out-of-possession tactical labels.
- The implementation consumes Layer 1 canonical SkillCorner outputs and writes reusable Layer 2 feature tables under data/processed/features/.
- It produces frame-level team shape metrics, phase-segment summaries, match-level tactical profiles, and a Layer 2 QA summary JSON.
- It supports both one-match processing and controlled batch processing across all processed SkillCorner matches.
- Schemas, metric sanity checks, batch processing, and phase-linking QA pass for all 10 processed SkillCorner matches.
- Orientation normalization exists only as a conservative/inactive module for now. Current analysis coordinates use the existing Layer 1 metric coordinates.
- This handoff does not authorize Layer 3 modeling, AI interpretation, dashboards, reports, recommendations, or advanced tactical claims.

2. Files Created

- src/soccer_tips/features/__init__.py
- src/soccer_tips/features/layer2_schemas.py
- src/soccer_tips/features/phase_context.py
- src/soccer_tips/features/orientation.py
- src/soccer_tips/features/team_shape.py
- src/soccer_tips/features/aggregation.py
- src/soccer_tips/features/build_layer2.py
- src/soccer_tips/features/qa_layer2_outputs.py
- tests/test_layer2_features.py

3. What Each File Does

File	Purpose
features/__init__.py	Defines the Layer 2 features package.
layer2_schemas.py	Defines schemas and schema helpers for frame team shape metrics, phase segment summaries, match tactical profiles, and Layer 2 QA output.

phase_context.py	Links SkillCorner Layer 1 tracking frames to phases_of_play.csv using frame_start, frame_end, and period. It determines in-possession versus out-of-possession status for each team-frame row.
orientation.py	Provides conservative analysis-coordinate helpers. Attacking-direction normalization is intentionally inactive in v0.1.
team_shape.py	Computes frame-level team shape metrics from x_metric/y_metric coordinates.
aggregation.py	Aggregates frame-level metrics into phase-segment summaries and match-level tactical profiles.
build_layer2.py	Command-line runner for one-match or batch Layer 2 feature generation.
qa_layer2_outputs.py	Checks output existence, schemas, metric sanity, and phase-linking behavior. Writes layer2_v01_qa_summary.json.
tests/test_layer2_features.py	Lightweight sanity tests for schema definitions, shape metric calculations, convex hull safety, and phase-linking examples.

4. How to Run Layer 2 v0.1

Open the terminal from the project root folder. On the current Windows setup, that project root is:

```
C:\Users\amsch\. Personal Work\Projects\Soccer TIPS
```

Commands below assume the VS Code integrated PowerShell terminal from the project root.

Task	PowerShell command
Activate virtual environment, if applicable	.\venv\Scripts\Activate
Install requirements	pip install -r requirements.txt
Set PYTHONPATH for src-layout imports	\$env:PYTHONPATH="src"
Build one SkillCorner match	python -m soccer_tips.features.build_layer2 --provider skillcorner --skillcorner-match-id 2017461
Build all processed SkillCorner matches	python -m soccer_tips.features.build_layer2 --provider skillcorner --batch

Recommended Run Order

1. Open PowerShell from the Soccer TIPS project root.
2. Activate the virtual environment if the project uses one.
3. Install requirements if needed.
4. Set PYTHONPATH to src.
5. Run the Layer 2 tests.
6. Run one SkillCorner match first.
7. Run the Layer 2 QA utility.
8. Run full SkillCorner batch mode.
9. Run the Layer 2 QA utility again.

```
.\venv\Scripts\Activate
pip install -r requirements.txt
$env:PYTHONPATH="src"
```

```
python tests/test_layer2_features.py
python -m soccer_tips.features.build_layer2 --provider skillcorner --skillcorner-match-id 2017461
python -m soccer_tips.features.qa_layer2_outputs
python -m soccer_tips.features.build_layer2 --provider skillcorner --batch
python -m soccer_tips.features.qa_layer2_outputs
```

5. How to Run Layer 2 Tests

```
$env:PYTHONPATH="src"
python tests/test_layer2_features.py
```

Expected result:

All Layer 2 feature sanity checks passed.

6. How to Run Layer 2 QA

```
$env:PYTHONPATH="src"
python -m soccer_tips.features.qa_layer2_outputs
```

Expected batch result after the accepted checkpoint:

Status: pass

7. Output Locations

Output	Location
Layer 2 feature outputs	data/processed/features/
Frame-level team shape metrics	data/processed/features/frame_team_shape_metrics.csv
Phase-segment summaries	data/processed/features/phase_segment_summaries.csv
Match tactical profiles	data/processed/features/match_tactical_profiles.csv
Layer 2 QA summary JSON	data/processed/features/layer2_v01_qa_summary.json

8. Output File Descriptions and Exact Row Grain

Output file	Description	Exact row grain
frame_team_shape_metrics.csv	Frame-level team shape metrics with phase context where available.	One row per provider, match_id, frame, and team_id. For SkillCorner v0.1 this generally means two team rows per usable tracking frame. The row represents a team-phase-frame snapshot. phase_index and phase label columns are populated only when the frame falls inside a SkillCorner phase interval.
phase_segment_summaries.csv	Aggregated summary of frame-level team metrics for phase segments.	One row per provider, match_id, phase_index, period, team_id, team_in_possession_id, in-possession phase label, out-of-possession phase

		label, and possession_status_for_team combination that has at least one linked frame-team metric row.
match_tactical_profiles.csv	Long-form match-level tactical profile table derived from frame-level metrics.	One row per provider, match_id, team_id, and profile scope. Current profile scopes include overall, possession_status, in_possession_phase, and out_of_possession_phase. Phase-specific profile rows are grouped by the relevant possession status and phase type.
layer2_v01_qa_summary.json	Structured QA report for the generated Layer 2 outputs.	One JSON object per QA run. It contains file checks, schema checks, metric checks, phase-linking diagnostics, warnings, and errors.

9. Metrics Currently Computed

- centroid_x: mean x_metric location of the team players used in the frame.
- centroid_y: mean y_metric location of the team players used in the frame.
- team_width: max y_metric minus min y_metric for the team in the frame.
- team_depth: max x_metric minus min x_metric for the team in the frame.
- convex_hull_area: area of the convex hull around team player coordinates when at least 3 valid player points exist.
- avg_distance_to_centroid: average Euclidean distance from each used player to the team centroid.
- player_count_used: number of valid player coordinate rows used for the team-frame calculation.
- detected_player_count: count of players with is_detected == True.
- extrapolated_player_count: count of players with is_detected == False.
- metric_warning_flag: Boolean flag for metric-quality warnings generated by the shape calculation layer.

10. Batch QA Results

Check	Accepted checkpoint result
Layer 2 tests	Pass
Single-match SkillCorner build for 2017461	Pass
Single-match Layer 2 QA	Pass
Batch SkillCorner build	Pass across all 10 processed SkillCorner matches
Batch Layer 2 QA	Pass
frame_team_shape_metrics.csv	873,296 rows x 21 columns, schema pass
phase_segment_summaries.csv	9,104 rows x 46 columns, schema pass
match_tactical_profiles.csv	394 rows x 21 columns, schema pass
negative_team_width_count	0
negative_team_depth_count	0
invalid_convex_hull_count	0
metric_warning_count	0
phase_linking status	Pass
linked_phase_row_count	609,614
missing_phase_context_count	263,682 rows
phase_linking_coverage_rate	Approximately 0.698
missing_inside_phase_interval_frame_count	0
missing_outside_phase_interval_frame_count	131,841 frames

11. Phase-Linking QA Interpretation

- The lower raw phase-linking coverage rate is not a linker failure.
- The key QA finding is that frames inside actual SkillCorner phase intervals had 0 failed links.
- Across the batch, `missing_inside_phase_interval_frame_count` was 0.
- The missing phase context rows are explained by tracking frames outside actual SkillCorner phase intervals, mostly gaps between phase windows.
- The QA utility was updated so it does not treat low raw phase coverage as a warning when all frames inside actual phase intervals link correctly.
- The current QA interpretation is: phase linking passes when no missing frame falls inside an actual SkillCorner phase interval. Low raw phase coverage can be acceptable when caused by gaps between phase intervals.
- Frame-level rows with missing phase context should not be used for phase-segment interpretation unless future logic explicitly assigns or fills phase context.

12. Known Limitations

- Layer 2 v0.1 is a feature-engineering foundation, not a completed tactical model.
- SkillCorner is the only active Layer 2 v0.1 provider for phase-aware processing.
- Metrica is intentionally not included in phase-aware Layer 2 v0.1 batch processing because it lacks the same validated `phases_of_play` structure.
- Orientation normalization is not active. The orientation module is conservative and currently preserves Layer 1 metric coordinates as analysis coordinates.
- Phase labels are only attached where SkillCorner `phases_of_play` intervals cover the tracking frame.
- The current outputs overwrite the existing Layer 2 output CSVs when the build runner is executed.
- The `match_tactical_profiles.csv` output is match-level and long-form; it is not yet a final cross-match team fingerprint dataset.
- No visual validation or football interpretation of the metric values has been completed yet.

13. What Remains Intentionally Out of Scope

- Layer 3 modeling.
- Clustering or unsupervised tactical identity modeling.
- AI interpretation or prompt-based tactical analysis.
- Dashboards.
- Reports or automated report generation.
- Recommendation systems.
- Advanced compactness interpretation.
- Transition stability modeling.
- Final tactical identity claims.
- Attacking-direction normalization.
- Cross-match team-level tactical fingerprints as a final analytical product.

14. Future Code-Helper Usage Note

Future code helpers should use this document together with:

- Engineering Department Doc, Validation & Data Ingestion Handoff Summary.
- Engineering Department Doc, Layer 1_v0.1 General Developer Handoff Notes.
- AIB Project System Architecture / Workflow Specification.
- This document explains what was built for Layer 2 v0.1 and how to run it.
- The validation handoff explains why the ingestion and phase-linking rules exist.
- The Layer 1 handoff explains how canonical processed inputs are produced.
- The architecture document explains how Layer 2 connects to the full Soccer TIPS system.
- Future helpers should not use this handoff alone to authorize Layer 3 modeling, AI interpretation, dashboards, reports, recommendations, or advanced tactical claims.

15. Scope Guard

- This handoff documents the accepted Layer 2 v0.1 feature-engineering foundation only.
- Do not add new Layer 2 metrics from this document alone.
- Do not begin compactness interpretation or transition stability modeling from this document alone.
- Before expanding metrics, use this document with the architecture document and obtain Lead Developer direction.
- Before moving beyond feature engineering into tactical modeling or interpretation, obtain Lead Developer and Project Manager review.